

Module 5 Structured Note: Reversing a String

1. What this module is about

Module 5 teaches how to reverse a C-style string.

Reversing a string means writing its characters in the opposite order.

Example:

```
python
```

Reversed:

```
nohtyp
```

The main skill in this module is learning two ways to reverse a string:

1. Reverse using an extra array
2. Reverse in-place using swapping

Both methods use the same important C-style string idea:

```
A[i] != '\0'
```

This means:

Keep moving through the string until the null terminator is found.

2. Required idea from previous modules

Before Module 5, remember this:

A C-style string is a character array that ends with `\0`.

Example:

```
char A[] = "python";
```

Memory view:

```
Index: 0 1 2 3 4 5 6  
Value: p y t h o n \0
```

The visible string is:

```
python
```

The `\0` is not part of the word. It only tells C where the string ends.

So when reversing:

```
p y t h o n
```

becomes:

```
n o h t y p
```

But `\0` must still remain at the end.

Correct reversed memory:

```
Index: 0 1 2 3 4 5 6  
Value: n o h t y p \0
```

Part A: Meaning of string reversal

3. What does reversing mean?

Reversing means the first character becomes the last character, the second character becomes the second-last character, and so on.

Example:

Original: python
Reversed: nohtyp

Character positions:

```
p -> last  
y -> second last  
t -> third last  
h -> third first  
o -> second first  
n -> first
```

So:

p y t h o n

becomes:

n o h t y p

4. The null terminator should not be reversed

A very important rule:

Do not treat `\0` as a normal character while reversing.

For this string:

p y t h o n \0

Only these characters are reversed:

p y t h o n

The `\0` must stay at the end of the final string.

Correct:

```
n o h t y p \0
```

Wrong idea:

```
\0 n o h t y p
```

If `\0` comes first, C thinks the string is empty because `\0` means “stop here.”

Part B: Method 1 — Reversing using an extra array

5. Main idea of the extra-array method

In this method, we create a second character array.

Example:

```
char A[] = "python";  
char B[10];
```

`A` stores the original string.

`B` will store the reversed string.

The idea is:

1. Go to the end of `A`.
 2. Move backward through `A`.
 3. Copy each character into `B` from the beginning.
 4. Add `\0` at the end of `B`.
-

6. Visual idea

Original string:

```
A = python
```

Memory:

```
Index: 0 1 2 3 4 5 6
A:      p y t h o n \0
```

Copy backward:

```
A[5] = n -> B[0]
A[4] = o -> B[1]
A[3] = h -> B[2]
A[2] = t -> B[3]
A[1] = y -> B[4]
A[0] = p -> B[5]
```

Then add:

```
B[6] = '\0';
```

Final:

```
B = nohtyp
```

7. Code for extra-array reversal

```
#include <stdio.h>

int main() {
    char A[] = "python";
    char B[10];
    int i, j;

    // Step 1: Move i to the end of the string
    for (i = 0; A[i] != '\0'; i++) {
        // empty loop
    }

    // Now i is at '\0'
    // Move i back to the last real character
    i = i - 1;

    // Step 2: Copy A backward into B forward
    for (j = 0; i >= 0; i--, j++) {
```

```

        B[j] = A[i];
    }

    // Step 3: Add null terminator at the end of B
    B[j] = '\0';

    printf("Original string: %s\n", A);
    printf("Reversed string: %s\n", B);

    return 0;
}

```

Output:

```

Original string: python
Reversed string: nohtyp

```

8. Understanding the first loop

The first loop is:

```

for (i = 0; A[i] != '\0'; i++) {
}

```

This loop has an empty body because its only job is to move `i` forward until it reaches `\0`.

For:

```
python
```

Memory:

```

Index: 0 1 2 3 4 5 6
Value: p y t h o n \0

```

Trace:

i	A[i]	Meaning
0	p	not end

i	A[i]	Meaning
1	y	not end
2	t	not end
3	h	not end
4	o	not end
5	n	not end
6	\0	stop

After the loop:

```
i = 6
```

But index `6` is `\0`, not a real character.

So we do:

```
i = i - 1;
```

Now:

```
i = 5
```

That means `i` points to the last real character:

```
A[5] = n
```

9. Understanding the second loop

The second loop is:

```
for (j = 0; i >= 0; i--, j++) {
    B[j] = A[i];
}
```

This means:

- `i` moves backward through `A`
- `j` moves forward through `B`
- each character from the end of `A` is copied into the beginning of `B`

Trace:

i	A[i]	j	B[j]
5	n	0	n
4	o	1	o
3	h	2	h
2	t	3	t
1	y	4	y
0	p	5	p

After this loop:

```
B = n o h t y p
```

But it is not yet a safe C-style string until we add `\0`.

So we write:

```
B[j] = '\0';
```

Now:

```
B = n o h t y p \0
```

10. Why `B[j] = '\0'` is necessary

This line is required:

```
B[j] = '\0';
```

Why?

Because `printf("%s", B)` prints characters until it finds `\0`.

If `\0` is missing, C does not know where the string ends.

Wrong result could look like:

```
nohtyp@#random...
```

This happens because C may continue reading memory after the array.

So after copying the reversed characters, always add:

```
B[j] = '\0';
```

Final safe result:

```
nohtyp\0
```

11. Dry run: extra-array method for `python`

Initial string:

```
A = python
```

Memory:

```
Index: 0 1 2 3 4 5 6  
A:      p y t h o n \0
```

Step 1: Find the end

```
for (i = 0; A[i] != '\0'; i++) {  
}
```

The loop stops when:

```
i = 6  
A[6] = \0
```

Then:

```
i = i - 1;
```

So:

```
i = 5
```

Now `i` points to `n`.

Step 2: Copy backward

Step	i	A[i]	j	B[j]
1	5	n	0	n
2	4	o	1	o
3	3	h	2	h
4	2	t	3	t
5	1	y	4	y
6	0	p	5	p

Step 3: Add `\0`

After copying:

```
j = 6
```

So:

```
B[6] = '\0';
```

Final memory:

```
Index: 0 1 2 3 4 5 6
B:      n o h t y p \0
```

Final output:

```
nohtyp
```

Part C: Method 2 — In-place reversal

12. Meaning of in-place reversal

In-place means:

Change the same array directly without creating another string array.

Example:

```
char A[] = "python";
```

Before:

```
A = python
```

After in-place reversal:

```
A = nohtyp
```

The same array `A` is modified.

13. Main idea of in-place reversal

Use two indexes:

```
i = left side
j = right side
```

For:

python

Memory:

Index: 0 1 2 3 4 5
Value: p y t h o n

Start:

i = 0
j = 5

Swap:

A[0] and A[5]
A[1] and A[4]
A[2] and A[3]

Then stop.

14. Code for in-place reversal

```
#include <stdio.h>

int main() {
    char A[] = "python";
    int i, j;
    char t;

    // Step 1: Move j to the end of the string
    for (j = 0; A[j] != '\0'; j++) {
        // empty loop
    }

    // Now j is at '\0'
    // Move j back to the last real character
    j = j - 1;
```

```
// Step 2: Swap characters from both ends
for (i = 0; i < j; i++, j--) {
    t = A[i];
    A[i] = A[j];
    A[j] = t;
}

printf("Reversed string: %s\n", A);

return 0;
}
```

Output:

```
Reversed string: nohtyp
```

15. Understanding the variable `t`

In this code:

```
t = A[i];
A[i] = A[j];
A[j] = t;
```

`t` is a temporary variable.

It helps us swap two characters safely.

Suppose:

```
A[i] = p
A[j] = n
```

If we directly write:

```
A[i] = A[j];
A[j] = A[i];
```

that would be wrong because the original value of `A[i]` is lost.

So we use `t`:

```
t = A[i];      // save p
A[i] = A[j];   // put n at the front
A[j] = t;      // put p at the end
```

After swap:

```
A[i] = n
A[j] = p
```

16. Dry run: in-place reversal for `python`

Initial string:

```
A = python
```

Memory:

```
Index: 0 1 2 3 4 5 6
A:      p y t h o n \0
```

Step 1: Find the last character

```
for (j = 0; A[j] != '\0'; j++) {
}
```

The loop stops at:

```
j = 6
A[6] = \0
```

Then:

```
j = j - 1;
```

So:

```
j = 5
```

Now `j` points to the last real character:

```
A[5] = n
```

Step 2: Swap from both ends

Start:

```
i = 0  
j = 5
```

Swap 1

Swap:

```
A[0] and A[5]  
p and n
```

Before:

```
p y t h o n
```

After:

```
n y t h o p
```

Now:

```
i = 1  
j = 4
```

Swap 2

Swap:

A[1] and A[4]
y and o

Before:

n y t h o p

After:

n o t h y p

Now:

i = 2
j = 3

Swap 3

Swap:

A[2] and A[3]
t and h

Before:

n o t h y p

After:

n o h t y p

Now:

i = 3
j = 2

The condition is:

```
i < j
```

But now:

```
3 < 2 is false
```

So the loop stops.

Final result:

```
nohtyp
```

Part D: Why the condition is `i < j`

17. Meaning of `i < j`

The loop condition is:

```
for (i = 0; i < j; i++, j--)
```

This means:

Keep swapping while the left index is before the right index.

When `i` and `j` meet or cross, the reversal is complete.

18. Even-length example

Example:

```
python
```

Length:

```
6
```

Indexes:

```
0 1 2 3 4 5
p y t h o n
```

Swaps:

```
0 with 5
1 with 4
2 with 3
```

After that, all characters are reversed.

There is no middle character.

Final:

```
nohtyp
```

19. Odd-length example

Example:

```
madam
```

Length:

```
5
```

Indexes:

```
0 1 2 3 4
m a d a m
```

Swaps:

```
0 with 4  
1 with 3
```

The middle character is:

```
d
```

The middle character does not need to move.

So the final string is still:

```
madam
```

This also shows why palindromes look the same after reversal.

Part E: Mutable arrays and string literals

20. Use a mutable character array for in-place reversal

For in-place reversal, use:

```
char A[] = "python";
```

This creates a character array that can be changed.

So this is safe for beginner code:

```
A[0] = 'n';
```

21. Avoid modifying string literals through pointers

This is risky:

```
char *A = "python";
```

Why?

Because `"python"` is a string literal.

A string literal may be stored in read-only memory.

Trying to change it can cause undefined behavior.

So this is not safe:

```
char *A = "python";  
A[0] = 'n'; // unsafe
```

Better:

```
char A[] = "python";  
A[0] = 'n'; // safe
```

Main rule:

If you want to change the string, store it in a character array.

Part F: Comparing both reversal methods

22. Extra-array method vs in-place method

Method	Main idea	Time complexity	Extra space
Extra array	Copy backward into another array	$O(N)$	$O(N)$
In-place	Swap characters from both ends	$O(N)$	$O(1)$

23. Extra-array method: advantages and disadvantages

Advantages:

- Easier to understand at first
- Original string stays unchanged
- Good for learning how copying works

Disadvantages:

- Uses another array

- Needs extra memory
- Must remember to add `\0` at the end of the new array

Example:

```
A = python
B = nohtyp
```

Here, `A` is still `python`.

24. In-place method: advantages and disadvantages

Advantages:

- Uses less extra memory
- Does not need a second string array
- Good for efficient string manipulation

Disadvantages:

- Changes the original string
- Requires careful swapping
- Needs a mutable character array

Example:

```
Before: A = python
After:  A = nohtyp
```

Here, the original `A` is changed.

Part G: Complexity analysis

25. Time complexity of extra-array reversal

The extra-array method scans the string to find the end.

That takes:

```
O(N)
```

Then it copies all characters backward.

That also takes:

$O(N)$

So total time is:

$O(N) + O(N) = O(N)$

Final time complexity:

$O(N)$

26. Space complexity of extra-array reversal

The extra-array method creates another array **B**.

If the input string has **N** characters, **B** also needs about **N** spaces.

So extra space complexity is:

$O(N)$

27. Time complexity of in-place reversal

The in-place method first scans the string to find the end.

That takes:

$O(N)$

Then it swaps characters.

For a string of length **N**, it performs about **$N / 2$** swaps.

But in Big-O notation:

$N / 2$ becomes $O(N)$

So final time complexity:

$O(N)$

28. Space complexity of in-place reversal

The in-place method uses only a few variables:

```
int i, j;  
char t;
```

It does not create another string array.

So extra space complexity is:

$O(1)$

This means constant extra space.

29. Complexity table

Operation	Time complexity	Extra space
Reverse using extra array	$O(N)$	$O(N)$
Reverse in-place	$O(N)$	$O(1)$

Part H: Common mistakes

30. Mistake 1: Reversing the null terminator

Wrong idea:

Reverse p y t h o n \0 completely

This could place \0 at the beginning.

Problem:

\0 n o h t y p

C would think the string ends immediately.

Correct idea:

Reverse only the visible characters.
Keep \0 at the end.

31. Mistake 2: Forgetting `B[j] = '\0'`

Wrong:

```
for (j = 0; i >= 0; i--, j++) {  
    B[j] = A[i];  
}
```

Problem:

`B` does not have a clear ending.

Correct:

```
for (j = 0; i >= 0; i--, j++) {  
    B[j] = A[i];  
}  
B[j] = '\0';
```

32. Mistake 3: Starting from \0 instead of the last character

After this loop:

```
for (i = 0; A[i] != '\0'; i++) {  
}
```

`i` points to `\0`.

So you must do:

```
i = i - 1;
```

Wrong:

```
// i still points to '\0'  
B[0] = A[i];
```

Correct:

```
i = i - 1;  
B[0] = A[i];
```

33. Mistake 4: Using `char *A = "python"` for in-place reversal

Wrong for modification:

```
char *A = "python";
```

This points to a string literal, which may not be safely changed.

Better:

```
char A[] = "python";
```

This creates a mutable array.

34. Mistake 5: Using the wrong loop condition for swapping

Correct condition:

```
i < j
```

Wrong idea:

```
i <= j
```

Using `i <= j` may make the middle character swap with itself in odd-length strings.

It usually does not change the result, but it is unnecessary.

The clean condition is:

```
i < j
```

because when `i` and `j` meet, the middle is already in the correct place.

Part I: Main code patterns to remember

35. Pattern 1: Find the end of a string

```
for (i = 0; A[i] != '\0'; i++) {  
}
```

After this loop, `i` is at `\0`.

To reach the last real character:

```
i = i - 1;
```

36. Pattern 2: Reverse using an extra array

```
for (i = 0; A[i] != '\0'; i++) {  
}  
  
i = i - 1;
```

```
for (j = 0; i >= 0; i--, j++) {  
    B[j] = A[i];  
}  
  
B[j] = '\0';
```

37. Pattern 3: Swap two characters

```
t = A[i];  
A[i] = A[j];  
A[j] = t;
```

This safely swaps `A[i]` and `A[j]`.

38. Pattern 4: Reverse in-place

```
for (j = 0; A[j] != '\0'; j++) {  
}  
  
j = j - 1;  
  
for (i = 0; i < j; i++, j--) {  
    t = A[i];  
    A[i] = A[j];  
    A[j] = t;  
}
```

Part J: Full combined examples

39. Full program: extra-array reversal

```
#include <stdio.h>  
  
int main() {  
    char A[] = "python";  
    char B[10];  
    int i, j;
```

```

    for (i = 0; A[i] != '\0'; i++) {
    }

    i = i - 1;

    for (j = 0; i >= 0; i--, j++) {
        B[j] = A[i];
    }

    B[j] = '\0';

    printf("Original string: %s\n", A);
    printf("Reversed string: %s\n", B);

    return 0;
}

```

Output:

```

Original string: python
Reversed string: nohtyp

```

40. Full program: in-place reversal

```

#include <stdio.h>

int main() {
    char A[] = "python";
    int i, j;
    char t;

    for (j = 0; A[j] != '\0'; j++) {
    }

    j = j - 1;

    for (i = 0; i < j; i++, j--) {
        t = A[i];
        A[i] = A[j];
        A[j] = t;
    }

    printf("Reversed string: %s\n", A);
}

```

```
    return 0;  
}
```

Output:

Reversed string: nohtyp

Part K: Practice questions

41. Practice 1

Reverse this string using the extra-array method:

python

Expected answer:

nohtyp

42. Practice 2

Reverse this string in-place:

code

Expected answer:

edoc

Swaps:

c with e
o with d

43. Practice 3

Reverse this odd-length string:

```
madam
```

Expected answer:

```
madam
```

Reason:

It is a palindrome, so it reads the same forward and backward.

44. Practice 4

What happens if we forget this line?

```
B[j] = '\0';
```

Answer:

`B` may not behave like a valid C-style string. `printf("%s", B)` may print extra garbage characters because it does not know where to stop.

45. Practice 5

Why should we use this for in-place reversal?

```
char A[] = "python";
```

Instead of this:

```
char *A = "python";
```

Answer:

`char A[] = "python";` creates a mutable character array. `char *A = "python";` points to a string literal, which may not be safely changed.

46. Practice 6

For the string:

hello

What are the swaps for in-place reversal?

Answer:

h with o
e with l
middle l stays in place

Final result:

olleh

Part L: Final summary

47. Key points to remember

- Reversing means writing characters in the opposite order.
- A C-style string ends with `\0`.
- The `\0` is not part of the visible word.
- Do not reverse the `\0` as a normal character.
- Extra-array reversal copies characters backward into a second array.
- Extra-array reversal must add `\0` at the end of the new array.
- In-place reversal swaps characters from both ends.
- In-place reversal uses two indexes: `i` from the left and `j` from the right.
- The swap loop continues while `i < j`.
- Use a temporary variable to swap safely.
- Use `char A[] = "python";` when modifying a string.
- Avoid modifying string literals through `char *A = "python";`.
- Extra-array reversal takes `O(N)` time and `O(N)` space.
- In-place reversal takes `O(N)` time and `O(1)` extra space.

48. Final mental model

For extra-array reversal:

```
Go to the end.  
Step backward.  
Copy into a new array.  
Add \0.
```

For in-place reversal:

```
Put one index at the start.  
Put one index at the end.  
Swap both characters.  
Move both indexes inward.  
Stop when they meet or cross.
```

49. Module 5 checkpoint

You are ready for Module 6 if you can do these without help:

1. Explain what string reversal means.
2. Explain why `\0` should not be reversed as a normal character.
3. Reverse a string using an extra array.
4. Explain why `B[j] = '\0'` is required.
5. Reverse a string in-place using two indexes.
6. Dry-run the in-place reversal of `python`.
7. Explain why the loop condition is `i < j`.
8. Explain why `char A[] = "python";` is safer than `char *A = "python";` for editing.
9. Compare `O(N)` space and `O(1)` space.
10. State the time and space complexity of both reversal methods.

Final takeaway:

Module 5 is about reversing a string by either copying it backward into a new array or swapping characters inside the same array. The most important detail is to keep the null terminator `\0` at the end so the result remains a valid C-style string.